

Model Fitting

Today's agenda:

- Model Fitting (7.1 - 7.2)

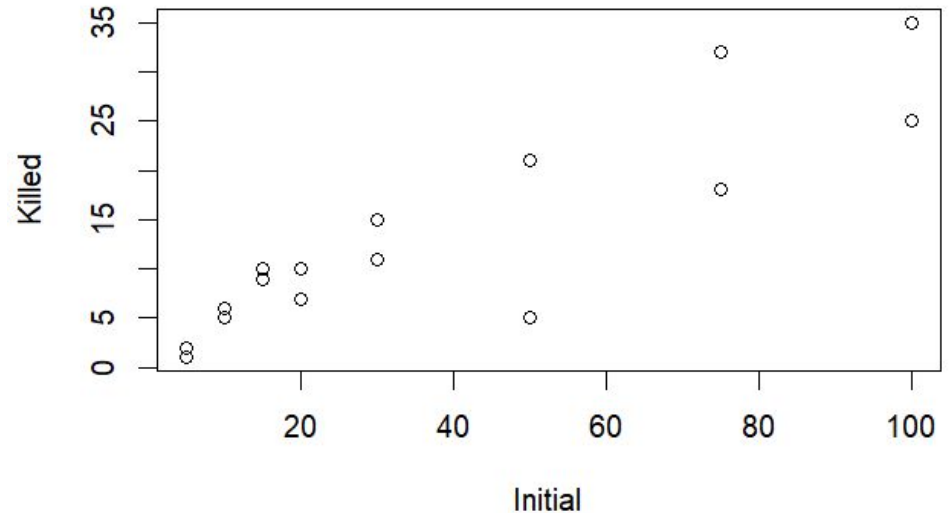
Previous classes

We've been talking about how to fit models

- Some folks have asked about the “method” argument in `optim` and `mle2`
- Today we'll cover what that is

Returning to the tadpole predation data

- Predation experiment
- Number killed as a function of initial number



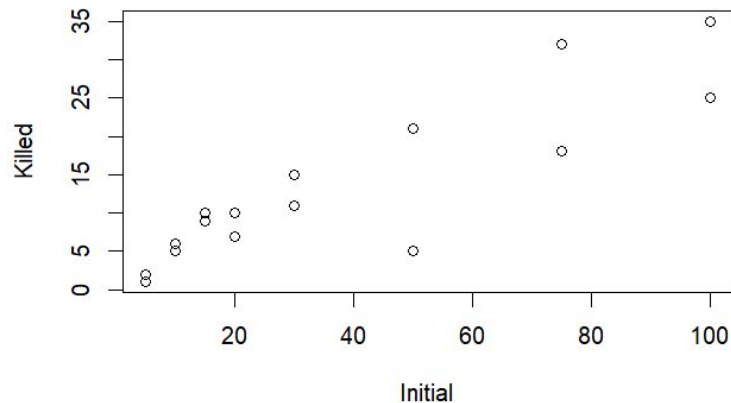
Modeling tadpole predation

Start by loading data and specifying a model

```
library(emdbook)
library(bbmle)
data("ReedfrogFuncresp")

binomNLL2 <- function(params, N, k){
  a = params[1]
  h = params[2]
  pred_prob = a / (1+a*h*N)

  -sum(dbinom(x = k, size = N, prob = pred_prob, log = TRUE))
}
```

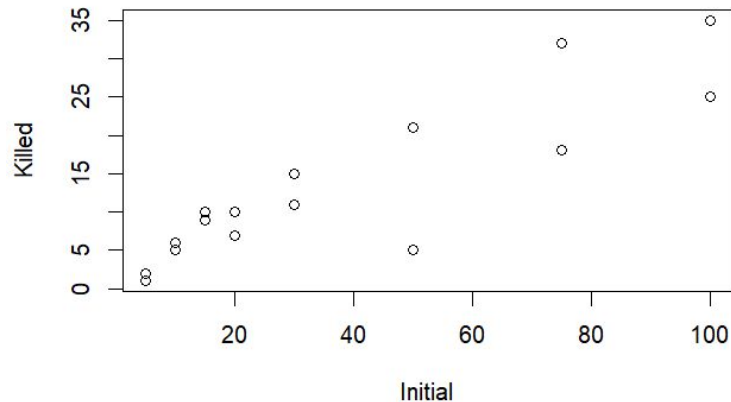


Modeling tadpole predation

Start by loading data and specifying a model

```
parnames(binomNLL2) <- c("a","h")
```

```
m1 <- mle2(minuslogl = binomNLL2,  
           start = c(a=.5,h=2),  
           data = list(k = ReedfrogFuncresp$Killed,  
                       N = ReedfrogFuncresp$Initial))
```



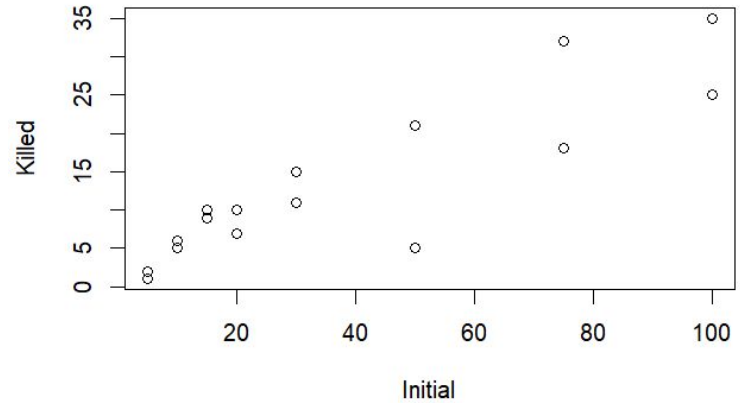
Modeling tadpole predation

Mle2 used an algorithm to fit your model

To find out which one, call

```
m1@method
```

What methods did it use?



Model fitting methods

R used a particular *algorithm* to fit your model

Algorithm: “a procedure for solving a mathematical problem”

But why does it need an algorithm at all?

Fitting models: Brute Force

Simplest way to fit a model: evaluate all possible parameters

- Doesn't require an algorithm
- But some serious drawbacks
- Let's try fitting our model using brute force

Brute force

To use brute force, we need to select parameters to check, let's try:

```
a_vec <- seq(from=0.1, to=0.9, by=.01)
h_vec <- seq(from=0.005, to=0.9, by=.001)

likelihood_surface <- expand.grid(a_vec,h_vec) %>%
  as.data.frame()
colnames(likelihood_surface) <- c("a","h")
likelihood_surface$nll <- NA
```

Take a look at `likelihood_surface` to make sure you understand what it contains

Brute force

Now, use a for() loop to evaluate the likelihoods:

```
for(i in 1:nrow(likelihood_surface)){  
  likelihood_surface$nll[i] <- binomNLL2(params = c(a = likelihood_surface$a[i],  
                                                    H = likelihood_surface$h[i]),  
                                          N = ReedfrogFuncresp$Initial,  
                                          k = ReedfrogFuncresp$Killed)  
}
```

Run this, and then find the best parameters using:

```
best_brute_force <- likelihood_surface[which.min(likelihood_surface$nll),]
```

How does this compare to your fit from m1?

Brute force

Your brute-force option gave similar (but not identical) results

- It only evaluates the options you told it to
- The differences can be much more severe in some cases
- If we had only evaluated “a” from 0.7 to 0.9 we’d have missed the best values!

You also may have notice that brute force was MUCH slower

- How much slower?

Timing your functions

`system.time()`

Will calculate the time it takes to run the code inside of it

Tells you:

User time: time for user-specified instructions to run

System time: time for system processes to run

Elapsed time : total time taken

Timing your functions

```
mle2_time <- system.time(  m1 <- mle2(minuslogl = binomNLL2,  
                                start = c(a=.5,h=2),  
                                data = list(k =ReedfrogFuncresp$Killed,  
                                              N=ReedfrogFuncresp$Initial)  
                                )  
                                )
```

Run this. How long did it take?

Timing your functions

```
brute_force_time <- system.time(  
  for(i in 1:nrow(likelihood_surface)){  
    likelihood_surface$nll[i] <- binomNLL2(params = c(a=likelihood_surface$a[i],  
                                                    h=likelihood_surface$h[i]),  
                                            N = ReedfrogFuncresp$Initial,  
                                            k = ReedfrogFuncresp$Killed)  
  }  
)
```

How long did this one take?

Timing your functions

Let's compare the two directly:

```
brute_force_time["elapsed"]/mle2_time["elapsed"]
```

Brute force is orders of magnitude slower!

Timing your functions

Brute force becomes even slower as we add more parameters.

- Time will scale exponentially with the number of parameters added!
- Brute force could take massive amounts of time
- For the same set of data, 6 parameters evaluated at 100 values each:
Roughly 2 years of run time!

Brute Force

- Slow
- Can miss the best option
- BUT can be useful when other options fail
- Usually, we rely on optimization algorithms

Optimization algorithms

Examples of optimization algorithms:

- 1) Derivative-based
 - a) BFGS
- 2) Derivative-free methods
 - a) Nelder-Mead Simplex
 - b) Simulated Annealing

BFGS

- Method by Broyden, Fletcher, Goldfarb and Shanno (1970)
- Derivatives tell us how the likelihood surface is changing
 - This method uses derivatives to figure out best values quickly
- Pros:
 - Fast
- Cons:
 - Fails when given bad starting values
 - Fails when likelihood surface is not smooth

BFGS

Let's modify our code to use BFGS:

```
m1.bfgs <- mle2(minuslogl = binomNLL2,  
               start = c(a=.5,h=2),  
               data = list(k = ReedfrogFuncresp$Killed,  
                           N = ReedfrogFuncresp$Initial),  
               method = "BFGS")
```

How do the results compare to the brute force fit or m1?

BFGS

What happens when we pick really bad parameters?

```
m1.bfgs.bad_start <- mle2(minuslogl = binomNLL2,  
                           start = c(a=.01,h=10),  
                           data = list(k = ReedfrogFuncresp$Killed,  
                                         N = ReedfrogFuncresp$Initial),  
                           method = "BFGS")
```

How do these estimates compare to the others?

Nelder-Mead

- Doesn't rely on derivatives
- Evaluates functions at multiple locations in each step
- Decides where to look and how far away to look based on local likelihoods
- The default method used by `optim` and `mle2`

Pros: More robust than BFGS

Cons: Slower than BFGS

Nelder-Mead

Let's try Nelder-Mead:

```
m1.nm <- mle2(minuslogl = binomNLL2,  
              start = c(a=.5,h=2),  
              data = list(k = ReedfrogFuncresp$Killed,  
                           N = ReedfrogFuncresp$Initial),  
              method = "Nelder-Mead")
```

This should give the same value as you got for m1

Nelder-Mead

What happens when we give this algorithm bad start values?

```
m1.nm.bad_start <- mle2(minuslogl = binomNLL2,  
                        start = c(a=.01,h=10),  
                        data = list(k = ReedfrogFuncresp$Killed,  
                                    N = ReedfrogFuncresp$Initial),  
                        method = "Nelder-Mead")
```

How do these parameters compare to `m1.nm`? How about `m1.bfgs.bad_start`?

Simulated Annealing

- Global optimizer
- Use stochasticity to sample surface (helps avoid local maxima/minima)
- Good choice for rugged surfaces

Pros:

- Very robust

Cons:

- Slow (but still **much** faster than brute force!)
- Can sometimes miss the best solution due to stochasticity
(particularly when given bad starting values)

Simulated Annealing

Let's try out Simulated Annealing

```
m1.sa <- mle2(minuslogl = binomNLL2,  
              start = c(a=.5,h=2),  
              data = list(k = ReedfrogFuncresp$Killed,  
                          N = ReedfrogFuncresp$Initial),  
              method = "SANN")
```

What parameter estimates did people get?

Simulated Annealing

Now give it bad starting values:

```
m1.sa.bad_start <- mle2(minuslogl = binomNLL2,  
                        start = c(a=.01,h=10),  
                        data = list(k = ReedfrogFuncresp$Killed,  
                                    N = ReedfrogFuncresp$Initial),  
                        method = "SANN")
```

How did it do?

Simulated Annealing and stochasticity

Let's use `set.seed` to make things a bit more reproducible and try again:

```
set.seed(1)
m1.sa.1 <- mle2(minuslogl = binomNLL2,
               start = c(a=.5,h=2),
               data = list(k = ReedfrogFuncresp$Killed,
                           N = ReedfrogFuncresp$Initial),
               method = "SANN")
```

```
set.seed(2005)
m1.sa.2005 <- mle2(minuslogl = binomNLL2,
                  start = c(a=.5,h=2),
                  data = list(k = ReedfrogFuncresp$Killed,
                              N = ReedfrogFuncresp$Initial),
                  method = "SANN")
```

Do the two functions give the same estimates?

Comparing our fits using AIC

Which of the model fit(s) did the best job?

Check using `AICtab()`

Which algorithms seem best in this case?

Take-aways

Algorithm	Pros	Cons
Brute Force	Simple	Slow, only selects among the selected values
BFGS	Fast	Fails with rough surfaces, relatively fragile
Nelder-Mead	More robust than BFGS, works with rough surfaces	Slower than BFGS
Simulated Annealing	Global optimizer, most robust to rough surfaces	Slower than Nelder-Mead and BFGS, stochasticity can cause issues

Before next class:

- Read 7.4